1 JavaScript Mastery Notes

1.1 Variables

Definition

Variable vaneko data store garne container ho. Javascript ma 3 ota variable declaration hunxa.

- Var : yo purano tarika ho , yo function-scoped hunxa jasko meaning hamiley function bitra matra simit hunxa vaney if and for loop ko block {} ley yeslai rokna sakdaina ani yesma Hoisting exist garxa jasko meaning javascript ley var bata baneko variable lai code chalnu aaagadi nai sabia vanda mathi purauxa tesailey declaration garnu aagafi nai print garda error aaudaina, undefined dekauxa ani lastly yesley redeclaration lai allow garxa.

Code:

```
if(true){
var x = 10;
}
console.log(x)
```

```
10                                                    VM291:4
undefined
```

Figure 1: var if-else bitra ra bahira case mah

```
for(var i = 1 ; i <= 3 ; i++ ){
    console.log("loop bitra ko value"+" "+ i)
}
    console.log("loop bahira"+ " " + i);
```

```
loop bitra ko value 1                                 VM602:2
loop bitra ko value 2                                 VM602:2
loop bitra ko value 3                                 VM602:2
loop bahira 4                                         VM602:4
undefined
```

Figure 2: var loop bitra ra bahira case mah

```
function demo(){
    var y = 10;
    console.log("function bitra" + "" + y) //yo kam garxa
}

demo();

console.log("function bahira " + " " + y) //yesma error
aauxa
```

```
function bitra10                                    VM1007:3
▶ Uncaught ReferenceError: y is not defined         VM1007:8
    at <anonymous>:8:40
```

Figure 3: var - function bitra ra bahira case mah

```
> var man = "manoj"
  console.log(man)
  var man = "calvin" // yo purei legal xa var ko case mah
  console.log(man)
```

```
manoj                                               VM678:2
calvin                                              VM678:4
< undefined
```

Figure 4: var ley redeclaration allow garxa

- Let : modern tarika, block-scoped hunxa tesko meaning function, if-else, loop , ko {} bahira acess garna mildaina tara reassign vaney garna milxa.

Example:

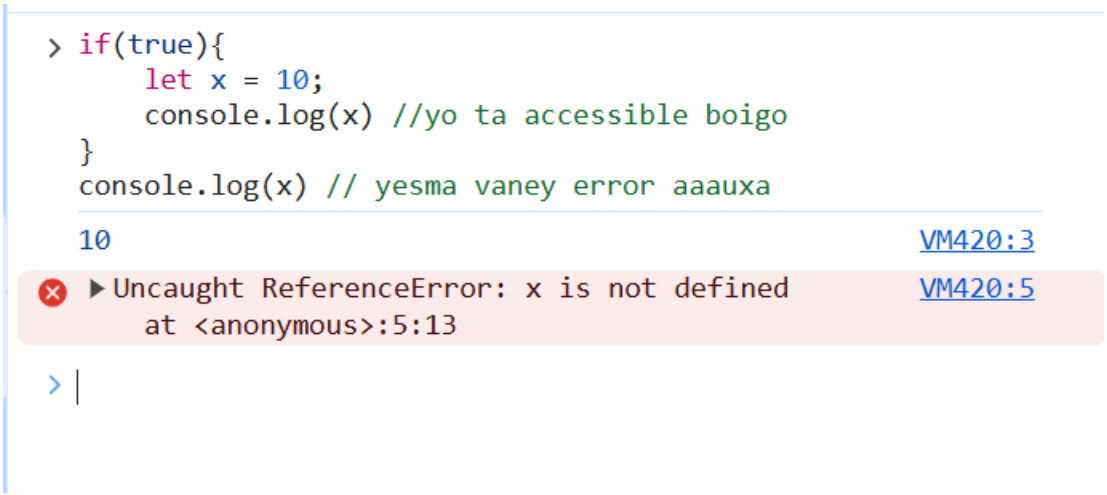


Figure 5: Let - if bitra bahira case mah

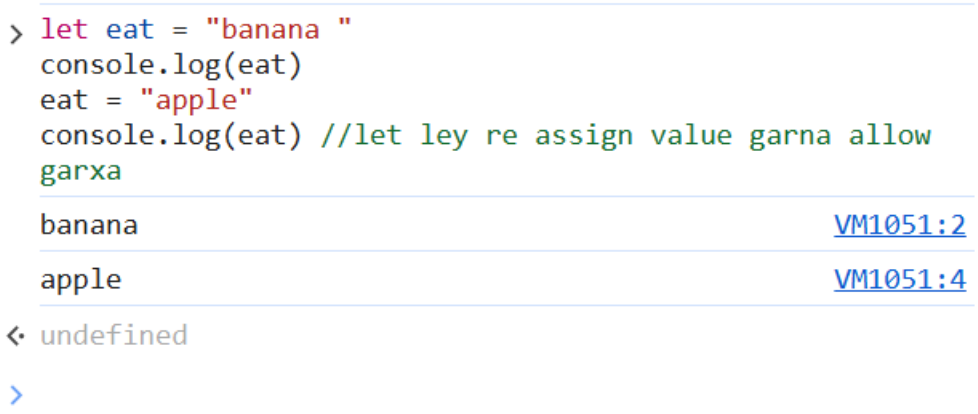
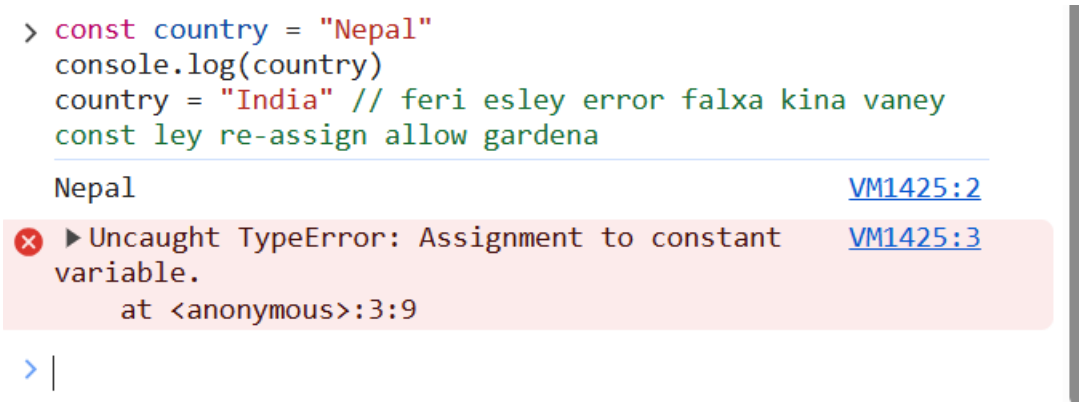


Figure 6: Let ley value re- assigned garna allow garxa

Const : block-scoped , reassign garna mildaina tara object array modify garna milxa.



2 Scope

Definition

Scope vaneko variable and function kaha access garna milxa vaney rule ho : Java script ma mukya tin ota scope hunxan ani tiniharu yes prakar ka hunxan :

- Global Scope : Sabei thau mah access garna sakiney variables.
- Function Scope: Function bitra declare gareko variables function bahira access garna sakidaina.
-
- Block scope = {} block bitra declare garika variables (let , const) block bahira access garna sakidaina.

Example

Global Scope

```
let globalVar = "I am global "
function demo(){
    console.log(globalVar)
}

demo();
console.log(globalVar)
```

```
I am global                                    VM505:3
I am global                                    VM505:7
undefined
```

Figure 7: Global Scope

Function Scope

```
function demo2(){
let functionVar = "I can access within the function"
    console.log(functionVar) // accesible here
}
demo2();
console.log(functionVar) // cannot be access
```

```
I can access within the function               VM931:3
▶Uncaught ReferenceError: functionVar is not   VM931:6
defined
    at <anonymous>:6:13
```

Figure 8: Function Scope

Block Scoped

```
> if(true){
      let blockVar = "I am block scope "
      console.log(blockVar) //Can be access from here
  }
  console.log(blockVar) //cannot be access outside from {}
  bracket

  I am block scope                                    VM1250:3

  ▶ Uncaught ReferenceError: blockVar is not          VM1250:5
  defined
      at <anonymous>:5:13

  >
```

Figure 9: Block Scoped

```
> function outer() {
    let outerVar = "Outer functin ley access garna sakxa ";
    function inner() {
      console.log(outerVar); // Accessible
    }
    inner();
  }
  outer();

  Outer functin ley access garna sakxa                VM1289:4

  undefined
```

Figure 10: Scope chain

3 Hoisting

Definition

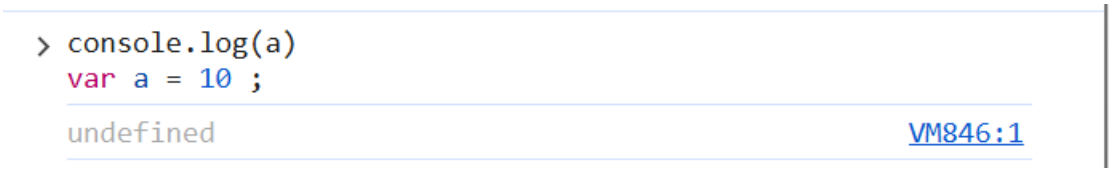
Hoisting vanaeko Javascript ko behavour ho jasma variable declaration (var ,let , const) and function declarations execution vanda pahila nai memory mah ujalinxa tara initialization value assign vaney ujalidana .

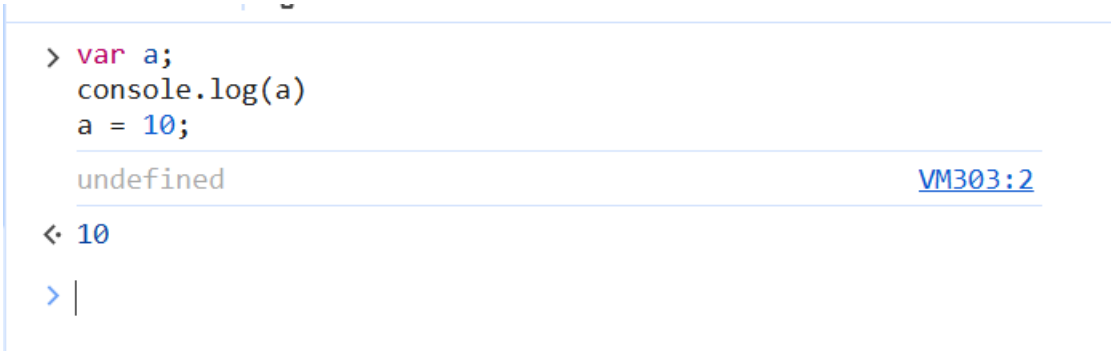
How it works:

- Var : hoisted with default value undefined.
- Let / const : hoisted tara Temporal Dead Zone (TDZ) mah rahanxan, declaration vanda pahila access garda error aaauxa.
- Function declaration: purna rupa hoisted hunxan, jasley garda function lai defina garnu banda aagadi call garna Sakixa.

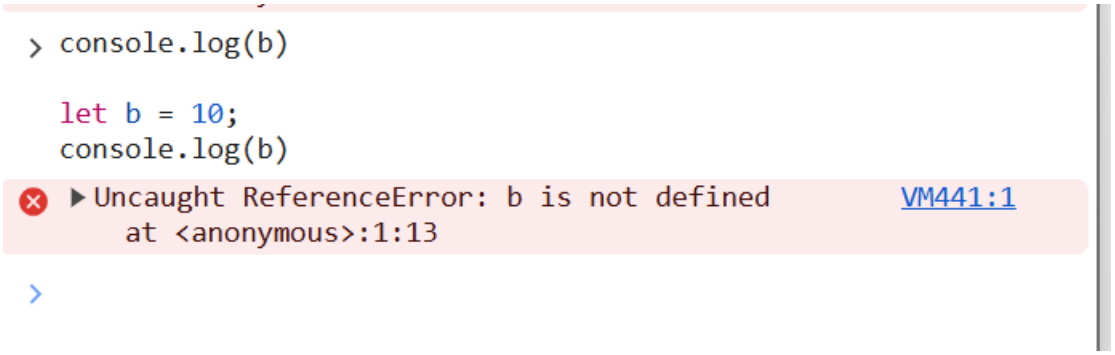
Example:

Var ko case ma herumlah





Yo mathikko code run garyo js ko engine ley talako code banauxa ani run hunxa tesari naii.



4 JavaScript Closures

4.1 Introduction

JavaScript मा **Closure** एउटा धेरै important concept हो। सुरुमा यो अलि difficult लाग्न सक्छ, तर once यो concept clear भयो भने React, Node.js, Express र modern JavaScript बुझ्न धेरै सजिलो हुन्छ।

Simple language मा भन्नुपर्दा,

Closure भनेको एउटा function हो जसले आफ्नो outer function को variables लाई remember गरिरहन्छ, even outer function execute भएर सकिएपछि पनि।

4.2 What is Closure?

Closure is a combination of:

- A Function
- Its Lexical Environment (Outer Scope)

यसको मतलब, एउटा function create हुँदा जुन variables हरू accessible थिए, त्यो function ले पछि पनि ती variables access गर्न सक्छ।

4.3 Why Do We Use Closures?

Closures प्रयोग गर्नुको मुख्य उद्देश्य भनेको function लाई previous values remember गराउनु हो। यसको मद्दतले private variables create गर्न सकिन्छ, data hide गर्न सकिन्छ, state maintain गर्न सकिन्छ र reusable functions बनाउन सकिन्छ।

Modern JavaScript frameworks जस्तै React, Node.js र Express मा closures धेरै प्रयोग गरिन्छ।

4.4 Basic Example

```
function outer() {
    let name = "Manoj";

    return function () {
        console.log(name);
    };
}

const showName = outer();

showName();
```

4.4.1 Output

```
Manoj
```

4.4.2 Explanation

यस example मा `outer()` function execute हुन्छ र `name` variable create हुन्छ। त्यसपछि inner function return हुन्छ। Outer function execute भएर सकिएपछि पनि inner function ले `name` variable लाई remember गरिरहेको हुन्छ। त्यसैले `showName()` call गर्दा **Manoj** print हुन्छ। यही behavior लाई **Closure** भनिन्छ।

4.5 Counter Example

```
function counter() {

    let count = 0;

    return function () {

        count++;

        console.log(count);

    };

}

const x = counter();

x();
x();
x();
```

4.5.1 Output

```
1
2
3
```

4.5.2 Explanation

पहिलो पटक `x()` call गर्दा `count` को value 1 हुन्छ। दोस्रो पटक call गर्दा 2 हुन्छ र तेस्रो पटक call गर्दा 3 हुन्छ। Outer function execute भएर सकिए पनि `count` variable memory मा सुरक्षित रहन्छ, किनकि returned function ले त्यसलाई अझै use गरिरहेको हुन्छ। यही Closure को मुख्य विशेषता हो।

4.6 Memory Representation

```
counter()

count = 0

↓

return function

↓

x()

count = 1

↓

x()

count = 2

↓

x()

count = 3
```

यस diagram ले देखाउँछ कि एउटै closure भित्र `count` variable continuously update भइरहेको छ।

4.7 Real-Life Example

Closure लाई एउटा locker को उदाहरणबाट सजिलै बुझ्न सकिन्छ।

- **Locker** = Outer Function

- **Important Documents** = Variables
- **Locker Key** = Returned Function

Owner बाहिर गए पनि जससँग locker को key छ, उसले documents access गर्न सक्छ। त्यसैगरी outer function execute भएर सकिएपछि पनि returned function ले outer function का variables access गर्न सक्छ।

4.8 Advantages of Closures

- Previous values remember गर्न मद्दत गर्छ।
- Private variables create गर्न सकिन्छ।
- Data hiding गर्न सकिन्छ।
- State maintain गर्न सकिन्छ।
- Function reusability बढाउँछ।
- Code organization राम्रो हुन्छ।

4.9 Disadvantages of Closures

- Extra memory use हुन सक्छ।
- धेरै closures प्रयोग गर्दा memory consumption बढ्न सक्छ।
- Complex applications मा debugging अलि difficult हुन सक्छ।

4.10 Real-World Applications

Closures धेरै ठाउँमा प्रयोग गरिन्छ, जस्तै:

- Event Handlers
- Callback Functions
- `setTimeout()`
- `setInterval()`
- React Hooks
- Express Middleware
- Node.js Applications
- Module Pattern

4.11 Conclusion

Closure JavaScript को एउटा powerful feature हो। यसले function लाई आफ्नो outer scope का variables remember गर्न मद्दत गर्छ, चाहे outer function execute भएर सकिएको किन नहोस्। Modern JavaScript development, विशेष गरी MERN Stack मा React Hooks, Event Handlers, Callbacks र Express Middleware जस्ता धेरै ठाउँमा closures प्रयोग हुने भएकाले यो concept राम्रोसँग बुझ्नु धेरै आवश्यक छ। Closure को राम्रो knowledge भए JavaScript programming अझ प्रभावकारी र professional रूपमा गर्न सकिन्छ।

5 JavaScript Prototype and Classes

5.1 Introduction

JavaScript मा **Prototype** र **Classes** Object-Oriented Programming (OOP) का धेरै important concepts हुन्। Prototype को मद्दतले objects ले methods र properties share गर्न सक्छन्, जसले memory बचाउँछ र code reusable बनाउँछ। ES6 पछि JavaScript मा **Class** introduce गरिएको हो, जसले Prototype को concept लाई अझ easy र readable बनाएको छ।

5.2 What is Prototype?

Prototype भनेको JavaScript को एउटा special object हो जसबाट अन्य objects ले properties र methods inherit गर्न सक्छन्।

जब कुनै object मा property वा method भेटिँदैन, JavaScript ले automatically त्यसको prototype मा खोज्छ। यदि prototype मा पनि भेटिएन भने, JavaScript ले prototype chain follow गर्दै माथिसम्म खोज्छ।

यस process लाई **Prototype Chain** भनिन्छ।

5.3 Why Do We Use Prototype?

Prototype प्रयोग गर्नुको मुख्य उद्देश्य एउटै method लाई धेरै objects ले share गर्न सकून् भन्ने हो।

यदि एउटै method प्रत्येक object भित्र create गरियो भने धेरै memory use हुन्छ। Prototype प्रयोग गर्दा method एकपटक मात्र create हुन्छ र सबै objects ले share गर्छन्।

यसले:

- Memory बचाउँछ।
- Code Reusability बढाउँछ।
- Performance राम्रो बनाउँछ।
- Code Maintain गर्न सजिलो हुन्छ।

5.4 Prototype Example

```
function Student(name) {
    this.name = name;
}

Student.prototype.sayHi = function () {
    console.log("Hi " + this.name);
};

const student1 = new Student("Manoj");

student1.sayHi();
```

5.4.1 Output

```
Hi Manoj
```

5.4.2 Explanation

यस example मा `Student` एउटा constructor function हो। `sayHi()` method प्रत्येक object भित्र create गरिएको छैन। यो `Student.prototype` मा create गरिएको छ। त्यसैले `student1` ले prototype बाट `sayHi()` method access गर्छ। यसले memory बचाउँछ किनकि एउटै method सबै objects ले share गर्छन्।

5.5 Prototype Chain

JavaScript ले property वा method खोज्दा तलको क्रम follow गर्छ।

```
Object

↓

Prototype
```

```
↓

Parent Prototype

↓

Object.prototype

↓

null
```

यदि object मा property भेटिएन भने Prototype मा खोजिन्छ। त्यहाँ पनि नभेटिए Parent Prototype मा खोजिन्छ। यस process लाई **Prototype Chain** भनिन्छ।

5.6 What is a Class?

Class भनेको object create गर्ने एउटा blueprint हो।

ES6 भन्दा अगाडि constructor function र prototype प्रयोग गरेर object create गरिन्थ्यो। ES6 पछि JavaScript ले Class introduce गर्यो, जसले Prototype को concept लाई अझ readable बनायो।

Class ले internally Prototype नै प्रयोग गर्छ।

5.7 Why Do We Use Classes?

Classes प्रयोग गर्दा:

- Code धेरै readable हुन्छ।
- Object create गर्न सजिलो हुन्छ।
- OOP concepts (Inheritance, Encapsulation, Polymorphism) implement गर्न सजिलो हुन्छ।
- Large projects मा code maintain गर्न easy हुन्छ।

5.8 Class Example

```
class Student {

    constructor(name) {
        this.name = name;
    }
```

```
    sayHi() {
        console.log("Hi " + this.name);
    }

}

const student1 = new Student("Manoj");

student1.sayHi();
```

5.8.1 Output

```
Hi Manoj
```

5.8.2 Explanation

यस example मा `constructor()` object create हुँदा automatically call हुन्छ। `this.name` ले object को name store गर्छ। `sayHi()` method prototype मा store हुन्छ र सबै Student objects ले एउटै method share गर्छन्।

5.9 Constructor

Constructor भनेको एउटा special method हो जुन object create हुँदा automatically execute हुन्छ।

Example:

```
constructor(name) {
    this.name = name;
}
```

यदि

```
const student = new Student("Manoj");
```

लेखियो भने constructor automatically call हुन्छ र `name` को value `"Manoj"` हुन्छ।

5.10 Prototype vs Class

Prototype	Class
Old Syntax	Modern Syntax

Prototype	Class
Constructor Function प्रयोग हुन्छ	class keyword प्रयोग हुन्छ
Prototype manually लेख्नुपर्छ	JavaScript automatically prototype use गर्छ
Readability कम हुन्छ	Readability धेरै राम्रो हुन्छ

5.11 Real-World Applications

Prototype र Classes धेरै ठाउँमा प्रयोग गरिन्छ।

- User Management System
- Employee Management System
- Banking System
- E-Commerce Website
- Student Management System
- React Class Components
- Node.js Applications
- Game Development

5.12 Advantages

- Memory Efficient
- Code Reusability
- Easy Maintenance
- Better Code Organization
- Supports Object-Oriented Programming
- Improves Readability

5.13 Disadvantages

- Prototype beginners लाई बुझ्न अलि difficult हुन सक्छ।
- Incorrect use गर्दा debugging difficult हुन सक्छ।
- Prototype Chain धेरै लामो भए performance मा सानो असर पर्न सक्छ।

5.14 Conclusion

Prototype JavaScript को core feature हो जसले objects लाई methods र properties share गर्न मद्दत गर्छ। ES6 Classes ले Prototype माथि cleaner र readable syntax प्रदान गर्छ। Modern JavaScript development मा हामी धेरैजसो Classes प्रयोग गर्छौं, तर internally JavaScript Prototype नै प्रयोग गरिरहेको हुन्छ। त्यसैले Prototype र Classes दुवै concept बुझ्नु MERN Stack developer बन्नका लागि धेरै आवश्यक हुन्छ।